

Stochastic Power Modeling and Constrained MDP Optimization for On-Device SLM Inference

Junfei Zhan

University of Pennsylvania
Philadelphia, USA
zjf2024@seas.upenn.edu

Weihang Ding

University of California, Berkeley
Berkeley, USA
dingcharles@berkeley.edu

Haoxun Shen

University of Pennsylvania
Philadelphia, USA
haoxun@seas.upenn.edu

Tengjiao He

Jinan University
Guangzhou, China
htj2018@jnu.edu.cn

Abstract—Deploying small language models (SLMs) on edge devices enables low-latency and privacy-preserving inference, but also introduces stringent energy and hardware constraints. We present a stochastic and interpretable framework for sustainable on-device SLM deployment. Fine-grained power measurements on representative models show distinct execution phases in CPU and GPU dynamics, motivating the use of Hidden Semi-Markov Models (HSMMs) to capture both long GPU plateaus and bursty CPU fluctuations. These models provide accurate predictions of energy and latency, serving as the foundation for a constrained Markov decision process (CMDP) that optimizes model and token configurations under finite battery budgets and instantaneous power caps. The CMDP integrates HSMM-based energy estimates with LLM-as-a-Judge quality scores to guide adaptive scheduling across inference tasks. Experiments on edge hardware demonstrate that HSMMs outperform Hidden Markov Model (HMM) and temporal convolutional network (TCN) baselines in power prediction, and that Q-learning in the CMDP improves average reward from about 9 to 15 while maintaining energy usage within 85–90% of the budget. This work provides the first unified mathematical framework linking SLM parameters, energy consumption, and inference quality, offering a principled tool for sustainable and adaptive deployment of AI services in edge networks and Internet of Things (IoT) systems.

Index Terms—on-device inference, small language models, hidden semi-Markov models, constrained Markov decision process, energy-quality trade-off

I. INTRODUCTION

Large language models (LLMs) have rapidly become the backbone of many applications in code generation, question answering, and human–AI interaction [1]–[3]. Traditionally, inference has been performed in cloud data centers equipped with powerful GPUs. However, privacy concerns, network costs, and latency issues are driving a paradigm shift: moving inference closer to the user, to edge devices such as smartphones, laptops, and IoT nodes [4], [5]. This trend promises low-latency and privacy-preserving services, but it also raises stringent challenges in terms of energy sustainability and hardware constraints [6]–[8].

When inference is executed on edge devices, small language models (SLMs) require a stable power supply, since each inference run is relatively long and accompanied by significant fluctuations in instantaneous power draw [9]. An unstable supply can even interrupt inference execution or degrade response quality [10], underscoring the need for careful power management. However, edge accelerators such as laptop GPUs

and embedded boards are bounded by finite battery capacity and device-level power caps [11], [12], making sustainable deployment of SLMs a significant technical challenge. This motivates the need for accurate modeling of SLM power consumption dynamics, so that resource-aware scheduling can balance energy usage against inference quality under strict hardware constraints.

Recent studies have begun treating energy as first-class metrics for deploying language models on edge devices. For instance, Camel [6] explores energy vs. latency trade-offs via hardware configuration (e.g. GPU frequency, batch size) on Jetson-like platforms; Husom et al. [7] provide systematic measurements of quantized SLMs across multiple datasets, focusing on energy and accuracy trade-offs. Edge-First Language Model Inference [13] further benchmarks SLM capabilities across edge and cloud, showing that inference workloads suffer greatly from resource bottlenecks when context lengths increase. However, these works generally assume each inference or task is isolated, static in configuration, and do not model the stochastic execution phases or setup overheads (such as model loading) that produce power spikes or variable energy drain over sequences of inference tasks.

This creates a fundamental trade-off in edge SLM deployment. On one hand, increasing model size, quantization fidelity improves inference output quality and robustness; on the other hand, these same dimensions increase computation, memory usage, and energy non-linearly, especially when accounting for initialization/setup phases and fluctuating CPU/GPU load [14]. Over sequences of inference tasks, these costs accumulate, making it essential not just to optimize single-task performance, but to schedule configurations task-by-task under battery/power constraints.

In this paper, we address the above gaps by presenting a data-driven and interpretable framework that jointly models the quality of the SLM inference, energy consumption, and power dynamics. Specifically: (i) we build Hidden Semi-Markov Models (HSMMs) to capture stochastic power and duration distributions over inference phases, (ii) we pair these power models with offline quality scoring (LLM-as-Judge) [15] to obtain cost-reward pairs for different SLM actions/configurations, (iii) we form a constrained Markov decision process (CMDP) over multiple sequential inference tasks with both a finite energy budget and device-level power

caps, and (iv) we solve CMDP with tabular Q-learning, obtaining adaptive policies that maximize cumulative inference quality while ensuring energy/power constraints are met.

II. ENERGY MODELING FRAMEWORK

A. Motivating Measurements

To understand the power dynamics during on-device inference, we conducted fine-grained measurements while running the MT-Bench dataset [15] locally using the *llama_cpp_python* backend with two representative SLMs: Qwen3-4B and Gemma2-2B. Fig. 1 illustrates the temporal evolution of CPU, GPU, and total device power consumption. As discussed below, these empirical observations reveal strong variability and piecewise-stationary phases, which motivate the need for a stochastic modeling framework.

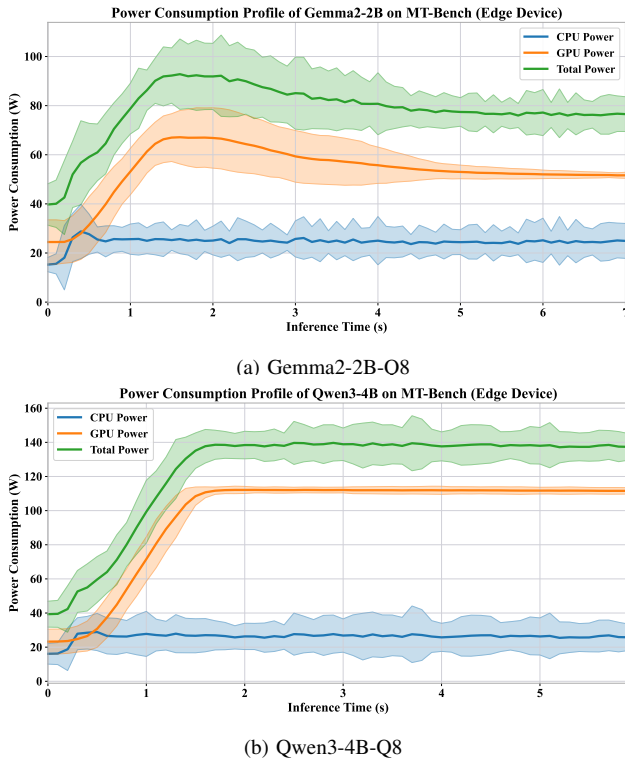


Fig. 1: Measured power consumption profiles on MT-Bench (edge device). (a) Gemma2-2B-Q8 and (b) Qwen3-4B-Q8. Power traces were collected using HWiNFO Pro with a sampling interval of 100 ms. Experiments were conducted on a laptop with Windows 10, NVIDIA RTX 3070 Laptop GPU, and Intel i7-10875H CPU.

From these measurements, we draw three key observations. First, the power traces exhibit clear *piecewise-stationary* phases, where CPU and GPU loads remain stable before transitioning to another regime, and the duration of each phase varies across models. Second, the overall trend of the total power is dominated by the GPU component, whereas short-term fluctuations are largely introduced by the CPU. Third, these phases are heterogeneous and time-dependent,

indicating that simple deterministic models are insufficient. Together, these observations motivate the use of a stochastic modeling framework, where CPU and GPU power are modeled separately with additive HSMMs: the GPU HSMM captures long plateau and ramp/decay behavior, while the CPU HSMM accounts for bursty fluctuations.

B. Hidden Semi-Markov Modeling of Edge Inference Energy

We decompose the device power into CPU and GPU contributions, motivated by the observation that GPU governs the long-term trend of the total power while CPU introduces short-term fluctuations. Thus, we model CPU and GPU processes with two independent HSMMs and reconstruct the total device power as their additive sum.

$$y_t^{\text{Tot}} = y_t^{\text{GPU}} + y_t^{\text{CPU}}. \quad (1)$$

We omit residual components such as memory and I/O power, since they remain nearly constant and negligible compared to CPU/GPU contributions during inference.

a) *State spaces*: Guided by Fig. 1, the GPU HSMM is represented with three hidden states (*ramp-up*, *high plateau*, and *decay*), while the CPU HSMM uses two hidden states (*low-load* and *high-load*). These parsimonious state spaces capture the distinct execution phases observed in practice. Fig. 2 illustrates the two parallel HSMMs, where solid arrows denote the dominant forward progressions and dashed arrows show reverse or self-transitions that are modeled to account for variability in empirical traces.

b) *HSMM definitions*: Each HSMM is specified by parameters

$$\Theta^X = \left\{ \pi^X, A^X, \{p_d^X(\cdot | k)\}_k, \{\mathcal{N}(\mu_k^X, \sigma_k^2)\}_k \right\}, X \in \{\text{CPU}, \text{GPU}\}. \quad (2)$$

where π^X is the initial state distribution, A^X the transition matrix, $p_d^X(\tau | k)$ the dwell-time distribution, and $\mathcal{N}(\mu_k^X, \sigma_k^2)$ the Gaussian emission for component X in state k .

c) *Observation models*: Conditioned on state $z_t^X = k$, the emission distribution is

$$y_t^X | z_t^X = k \sim \mathcal{N}(\mu_k^X, \sigma_k^2). \quad (3)$$

The total power is then obtained by summing the two modeled contributions.

d) *Duration models*: Unlike standard HMMs, each HSMM explicitly characterizes the probability of remaining in state k for τ steps:

$$p_d^X(\tau | k) = \Pr(D_t^X = \tau | z_t^X = k), \quad \tau \in \mathbb{N}^+, \quad (4)$$

where D_t^X is the state sojourn time for component X .

e) *Learning*: The parameters Θ^{CPU} and Θ^{GPU} are learned independently from measured traces via the Expectation-Maximization (EM) algorithm, with HSMM forward-backward recursions augmented by duration distributions. After training, the total power sequence can be reconstructed or predicted by superimposing the

expected CPU and GPU contributions, thereby achieving an interpretable and accurate representation of on-device inference energy dynamics.

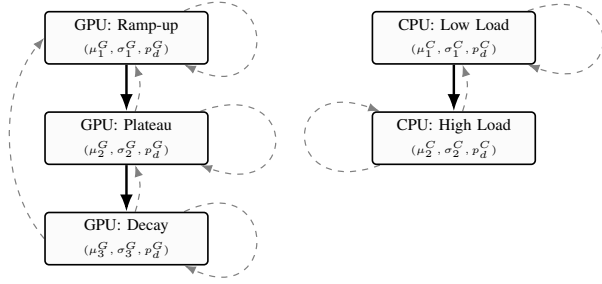


Fig. 2: Two parallel HSMMs for on-device inference power. Left: GPU (ramp-up, plateau, decay); Right: CPU (low/high). Solid arrows: dominant transitions; dashed arrows: reverse/self transitions.

III. CONSTRAINED MARKOV DECISION PROCESS FOR MULTI-TASK EDGE INFERENCE

Building on the HSMM-based energy models from Section II, we now formulate a CMDP to capture the trade-off between inference quality and energy consumption on edge devices under realistic constraints. The central question is: given multiple SLMs, which setup should be deployed for each task so as to maximize cumulative inference quality while respecting both a finite battery budget and a device-level power cap?

A. Problem Setup

We consider a sequence of T inference tasks. Before task i starts, the agent selects an action $a_i = (m_i, q_i)$, where m_i indexes the SLM variant (e.g., *Dolly-3B*, *Gemma-2B*, *Qwen-4B*) and q_i denotes the quantization scheme (e.g., Q6/Q8). The chosen action remains fixed throughout the task. The system state at task i is simply $s_i = b_i$, where b_i is the remaining battery energy before executing task i .

B. State, Action, and Transition Dynamics

Given an action a_i , we use the CPU and GPU HSMMs fitted for that configuration to obtain the expected energy consumption:

$$\mathbb{E}[E(a_i)] = \sum_k \mu_k^G(a_i) \Delta t \mathbb{E}[D_k^G(a_i)] + \sum_j \mu_j^C(a_i) \Delta t \mathbb{E}[D_j^C(a_i)]. \quad (5)$$

where μ_k^X and D_k^X are the mean power and sojourn time of phase k in the HSMM of component $X \in \{G, C\}$. These HSMM-based estimates provide the per-task cost signal for the CMDP. The battery then evolves deterministically in expectation:

$$b_{i+1} = b_i - \mathbb{E}[E(a_i)], \quad b_1 = B_0. \quad (6)$$

C. Reward Function and Safety Constraints

We use a scalar quality score as the reward:

$$r(s_i, a_i) = \hat{q}(a_i), \quad (7)$$

where $\hat{q}(\cdot)$ denotes the quality score obtained with GPT-4o in the *LLM-as-a-Judge* setting, following the MT-Bench Arena protocol [15]. In addition to energy budgeting, we impose a device-level power cap $P_{\max}$. From the fitted HSMMs, we extract the phase-wise distributions of instantaneous power and enforce feasibility by discarding actions that violate the cap:

$$\mathcal{A}_{\text{cap}} = \{a : \max_k Q_{1-\delta}^{(k)}(a) \leq P_{\max}\}, \quad (8)$$

where $Q_{1-\delta}^{(k)}$ is the $(1 - \delta)$ -quantile of the total power distribution in phase k (Gaussian case: $Q_{1-\delta}^{(k)} = \mu_k^{\text{Tot}} + z_{1-\delta} \sigma_k^{\text{Tot}}$).

D. Optimization Objective and Bellman Recursion

The CMDP formulation seeks to maximize cumulative quality while constraining both energy and instantaneous power.

$$\begin{aligned} \max_{\pi} \quad & \mathbb{E}_{\pi} \left[\sum_{i=1}^T r(s_i, a_i) \right] \\ \text{s.t.} \quad & \sum_{i=1}^T \mathbb{E}[E(a_i)] \leq B_0, \\ & a_i \in \mathcal{A}_{\text{cap}}, \quad \forall i = 1, \dots, T. \end{aligned} \quad (9)$$

Let $V_i(b)$ denote the maximum expected quality achievable from the task i with the remaining energy b . The Bellman recursion is

$$\begin{aligned} V_i(b) &= \max_{a \in \mathcal{A}_{\text{cap}}: \mathbb{E}[E(a)] \leq b} \left\{ \hat{q}(a) + V_{i+1}(b - \mathbb{E}[E(a)]) \right\}, \\ V_{T+1}(b) &= 0. \end{aligned} \quad (10)$$

IV. EVALUATION

All experiments were conducted on a laptop running Windows 10 with an NVIDIA RTX 3070 Laptop GPU and an Intel i7-10875H CPU. Power measurements were collected using HWiNFO Pro. Table I summarizes the inference defaults, CMDP constraints, and Q-learning setup. The purpose of this evaluation is to demonstrate that our HSMM-driven CMDP formulation enables (i) accurate modeling of action-level energy cost, (ii) a realistic energy–quality trade-off across candidate SLM actions, and (iii) effective policy learning that respects both budget and power-cap constraints.

TABLE I: Experimental configuration for CMDP evaluation

Inference Defaults	N_GPU_LAYERS=48, MAX_TOKENS=250, TEMPERATURE=0.7, TOP_P=0.95, TOP_K=40, N_THREADS=None, N_CTX=4096
CMDP Constraints	Energy budget $B_0 = 2400$ J, Power cap $P_{\max} = 140$ W
Q-learning Setup	Horizon $T = 10$; episodes 300; learning rate $\alpha = 0.1$; discount $\gamma = 1.0$; ϵ -greedy: $1.0 \rightarrow 0.05$

Cost modeling with HSMM: We model per-action energy via phase-wise CPU/GPU HSMMs trained on device power traces. As benchmarks, we compare against a standard HMM and a temporal convolutional network (TCN) regression baseline. Fig. 3 shows an example on *Qwen-4B (Q6)*, where HSMM most accurately tracks held-out CPU and GPU power, capturing both state durations and variance. This validates HSMM as the cost model used in CMDP optimization.

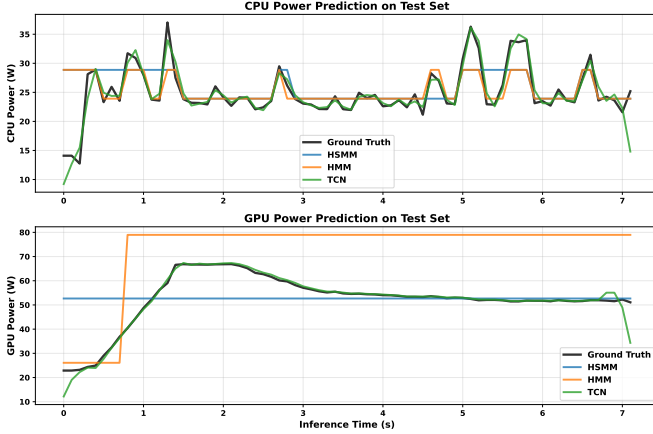


Fig. 3: Example prediction on held-out traces of *Qwen-4B (Q6)*. HSMM achieves the closest fit for both CPU (top) and GPU (bottom) compared to HMM and TCN.

Action space: Based on HSMM-predicted energy and offline MT-Bench quality, we construct the cost–reward pairs for six candidate actions (*Dolly-3B*, *Gemma-2B*, *Qwen-4B*, each with Q6/Q8). Table II provides these values, which serve as the inputs to CMDP optimization and a reference for edge-side trade-offs.

TABLE II: Action space: HSMM-predicted energy and MT-Bench quality for each candidate action.

Action	Energy (J)	Quality Score
Dolly-3B (Q6)	~514	4.5
Dolly-3B (Q8)	~587	4.9
Gemma-2B (Q6)	~607	6.7
Gemma-2B (Q8)	~643	6.4
Qwen-4B (Q6)	~548	5.0
Qwen-4B (Q8)	~711	6.3

Policy learning and constraint satisfaction: We train a tabular Q-learning agent with horizon $T=10$ using the above cost–reward pairs. Fig. 4 shows that the average reward increases from about 8 at initialization to 15–16 after roughly 200 episodes, representing nearly a two-fold improvement. Meanwhile, the energy usage ratio remains consistently below the budget, converging to around 0.85–0.90 without ever exceeding the limit. These results demonstrate both the effectiveness of policy learning and the strict satisfaction of CMDP constraints without additional on-device sampling.

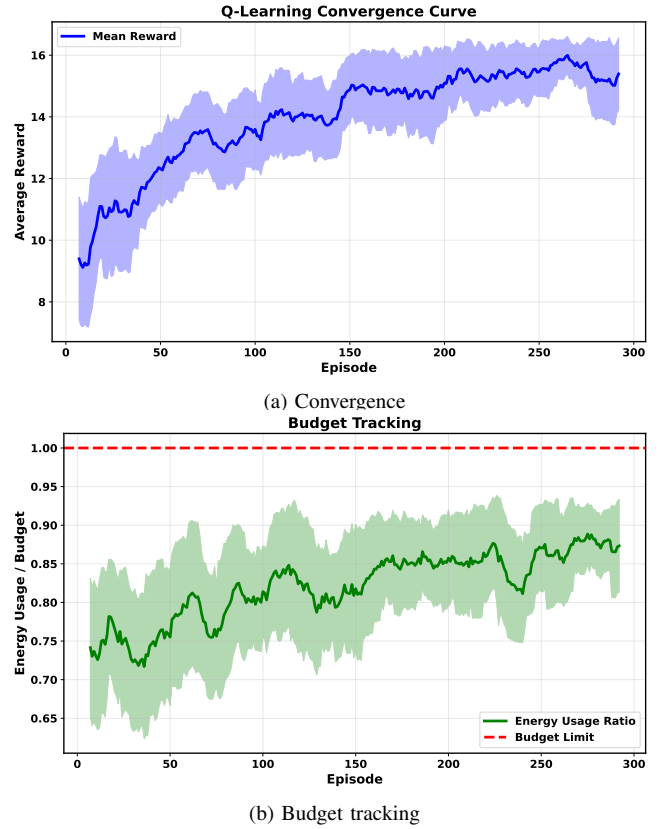


Fig. 4: Policy learning in the HSMM-driven CMDP. (a) Average reward converges within a few hundred episodes. (b) Energy usage remains below the budget throughout training.

V. CONCLUSION

This paper introduced a stochastic and interpretable framework for sustainable on-device deployment of SLMs. We first proposed parallel HSMMs to capture CPU and GPU power dynamics during inference, showing superior fidelity over HMM and TCN baselines. These models provided accurate action-level cost estimates, which were then combined with offline quality scores to construct cost–reward pairs for candidate SLM configurations. Building on this, we formulated a CMDP over multiple inference tasks and solved it with tabular Q-learning. Evaluation on representative SLMs demonstrated that (i) HSMM-based energy predictions align closely with measured traces, (ii) the resulting action space exposes clear energy–quality trade-offs, and (iii) learned CMDP policies achieve substantial quality gains while consistently respecting energy and power constraints.

Future directions include scaling the CMDP formulation to deep reinforcement learning with richer state representations, extending to heterogeneous edge clusters with collaborative budgeting, and integrating latency constraints alongside energy and power considerations. We believe our framework provides a principled foundation for sustainable and adaptive deployment of SLMs on resource-constrained edge devices.

REFERENCES

- [1] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr *et al.*, “Flamingo: A visual language model for few-shot learning,” in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/960a172bc7fb0177ccccbb411a7d800-Paper-Conference.pdf
- [2] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [3] I. Drori, S. Zhang, R. Shuttlesworth, L. Tang, A. Lu *et al.*, “A neural network solves, explains, and generates university math problems by program synthesis and few-shot learning at human level,” *Proceedings of the National Academy of Sciences*, vol. 119, no. 32, 2022. [Online]. Available: <https://doi.org/10.1073/pnas.2123433119>
- [4] Z. Li, W. Feng, M. Guizani, and H. Yu, “Tpi-llm: Serving 70b-scale llms efficiently on low-resource mobile devices,” *IEEE Transactions on Services Computing*, pp. 1–13, 2025.
- [5] A. Khoshsirat, G. Perin, and M. Rossi, “Decentralized llm inference over edge networks with energy harvesting,” in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 3703–3708.
- [6] H. Xu, L. Peng, S. Song, X. Liu, M. Jun, S. Li, J. Yu, and X. Mao, “Camel: Energy-aware llm inference on resource-constrained devices,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.09173>
- [7] E. J. Husom, A. Goknil, M. Astekin, L. K. Shar, A. Kâsen, S. Sen, B. A. Mithassel, and A. Soyly, “Sustainable llm inference for edge ai: Evaluating quantized llms for energy efficiency, output accuracy, and inference latency,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.03360>
- [8] C. Tian, X. Qin, K. Tam, L. Li, Z. Wang, Y. Zhao, M. Zhang, and C. Xu, “Clone: Customizing llms for efficient latency-aware inference at the edge,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.02847>
- [9] A. Pinnock, S. Jayakody, K. A. Roxy, and M. R. Ahmed, “Edgeprofiler: A fast profiling framework for lightweight llms on edge using analytical model,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.09061>
- [10] M. Arya and Y. Simmhan, “Understanding the performance and power of llm inferencing on edge accelerators,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.09554>
- [11] G. Xu, Z. Hao, Y. Luo, H. Hu, J. An, and S. Mao, “Devit: Decomposing vision transformers for collaborative inference in edge devices,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5917–5932, 2024.
- [12] M. Kasioulis, M. Symeonides, G. Ioannou, G. Pallis, and M. D. Dikaiakos, “Energy modeling of inference workloads with ai accelerators at the edge: A benchmarking study,” in *2024 IEEE International Conference on Cloud Engineering (IC2E)*, 2024, pp. 189–196.
- [13] S. Jang and R. Morabito, “Edge-first language model inference: Models, metrics, and tradeoffs,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.16508>
- [14] G. Wilkins, S. Keshav, and R. Mortier, “Offline energy-optimal llm serving: Workload-based energy models for llm inference on heterogeneous systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.04014>
- [15] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.05685>